Anatomía de un LLM

Fundamentos matemáticos de los modelos de lenguaje

Fase IV · Escala, sistemas y arquitecturas modernas

Módulo 16

Preentrenamiento a escala

Resumen del módulo

Edición de octubre de 2026

Sobre este resumen

Este documento reúne lo indispensable de los apuntes del Módulo 16: el texto expositivo, todas las definiciones y todos los resultados (teoremas, proposiciones, lemas y corolarios), las tablas y las observaciones de contenido. Se han suprimido las demostraciones, los ejemplos, las lecturas, las figuras, los ejercicios y sus soluciones. La numeración de temas, secciones, enunciados y ecuaciones es la misma que en los apuntes, de modo que cualquier referencia puede buscarse allí; las que remiten a elementos suprimidos (un ejemplo, una figura, una ecuación de una demostración) conservan su número en los apuntes.

Notación

Símbolo	Significado
N, D, C	parámetros, tokens de entrenamiento y FLOPs de entrenamiento
L, $L(N, D)$	pérdida de validación (nats por token) y su ley de escalado
E, A, B, α, β	constantes de la ley de Chinchilla
$N_{\mathrm{opt}}(C)$, $D_{\mathrm{opt}}(C)$	reparto óptimo para el presupuesto C
w_i, U_i, e_i	peso, tokens únicos y épocas de la fuente i de una mezcla
$J(A, B)$	similitud de Jaccard de dos conjuntos
h, $\mathrm{sig}(S)$	función hash y firma MinHash del conjunto S
b, r	bandas y filas por banda del LSH ($n = br$ funciones hash)
k	longitud de una respuesta en tokens o de un *shingle*
n, n_{ctx}	longitud de la secuencia y ventana de contexto
ln, $\log_{10}$	logaritmo natural y decimal

Las pérdidas se miden en nats por token. Se usa la escala numérica larga del español: un *billón* es 10^{12} y mil millones, 10^9.

Índice

Sobre este resumen **ii**

1 Datos de preentrenamiento **1**
1.1 Del rastreo web al corpus . . . 1
1.2 Filtros heurísticos de calidad . . . 1
1.3 Filtro por perplejidad . . . 2
1.4 Clasificadores de calidad . . . 2
1.5 Mezcla de fuentes y épocas . . . 3
1.6 La escala de los datos . . . 3

2 Deduplicación: MinHash y LSH **4**
2.1 Por qué deduplicar . . . 4
2.2 Duplicados exactos . . . 4
2.3 Shingles y similitud de Jaccard . . . 5
2.4 Firmas MinHash . . . 5
2.5 LSH por bandas . . . 5

3 Leyes de potencia y leyes de escalado de Kaplan **7**
3.1 Leyes de potencia . . . 7
3.2 Ajuste en escala log-log . . . 7
3.3 Las leyes de Kaplan . . . 8
3.4 Ley conjunta y sobreajuste . . . 8
3.5 El reparto de Kaplan . . . 9

4 El reparto óptimo del cómputo: Chinchilla **10**
4.1 El problema del reparto . . . 10
4.2 La ley paramétrica de Chinchilla . . . 10
4.3 La solución en forma cerrada . . . 11
4.4 Gopher frente a Chinchilla . . . 11
4.5 La regla de los 20 tokens . . . 11
4.6 El mínimo es plano . . . 11
4.7 Forma convexa del problema . . . 12

5 Estimación empírica, inferencia y datos limitados **13**
5.1 Tres métodos de estimación . . . 13
5.2 Por qué difieren Kaplan y Chinchilla . . . 14
5.3 El coste de la inferencia . . . 14
5.4 Datos limitados: el valor de repetir . . . 15

6 Capacidades emergentes y contexto largo **16**
6.1 Capacidades emergentes . . . 16
6.2 El argumento de la métrica . . . 16
6.3 El coste del contexto largo . . . 17
6.4 Posiciones no vistas: interpolación de RoPE . . . 18
6.5 La receta habitual y sus límites . . . 18

Bibliografía **19**

Tema 1

Datos de preentrenamiento

El preentrenamiento minimiza la entropía cruzada media sobre un corpus (módulo 1, tema 5), de modo que el modelo aprende la distribución del texto que se le muestra. La composición de ese corpus es, por tanto, una decisión de diseño tan importante como la arquitectura. Este tema describe cómo se obtiene un corpus a partir de la web y formaliza tres de sus etapas: los filtros de calidad, el filtro por perplejidad y la mezcla de fuentes. Se siguen las secciones 2 y 4 de la práctica de datos de CS336 [10], la clase sobre datos de CS324 [16] y los informes de CCNet [28], Gopher [22, §3.4 y apéndice A], RefinedWeb [19, §3], FineWeb [20] y LLaMA [25, §2.1]. La deduplicación, que también forma parte del proceso, se estudia por separado en el tema 2.

1.1 Del rastreo web al corpus

Definición 1.1 (Corpus de preentrenamiento). Un *corpus de preentrenamiento* es una colección finita de documentos $\mathcal{C} = \{x^{(1)}, \ldots, x^{(M)}\}$, cada uno una secuencia de tokens. Su *tamaño* es el número total de tokens, $\sum_m |x^{(m)}|$. Un *filtro* es un predicado f sobre documentos; aplicarlo a $\mathcal{C}$ produce $\{x \in \mathcal{C} : f(x) = 1\}$. Un proceso de preparación es una composición de transformaciones (extracción, normalización) y filtros.

La fuente principal es la web. Common Crawl publica volcados periódicos de miles de millones de páginas; FineWeb, por ejemplo, se construyó a partir de 96 volcados y contiene unos 15 billones de tokens [20]. La figura 1.1 resume las etapas habituales. Casi todo el material en bruto se descarta: en RefinedWeb, la identificación del idioma elimina cerca de la mitad de los documentos, y el conjunto del proceso, casi el 90 % [19, §3 y figura 2].

1.2 Filtros heurísticos de calidad

Los filtros más sencillos son reglas sobre estadísticas superficiales del documento. Son baratos, deterministas y fáciles de auditar. El informe de Gopher [22, apéndice A] define un conjunto de ellas; la práctica de CS336 [10, §2.6] usa el siguiente subconjunto.

Definición 1.2 (Filtros de Gopher, subconjunto de CS336). Un documento con W palabras, longitud media de palabra $\bar{\ell}$ (en caracteres) y Λ líneas *pasa* el filtro si se cumplen las cuatro condiciones siguientes:

(i) $50 \le W \le 100\,000$;

(ii) $3 \le \bar{\ell} \le 10$;

(iii) como mucho el 30 % de las Λ líneas termina en puntos suspensivos;

(iv) al menos el 80 % de las palabras contiene un carácter alfabético.

1.3 Filtro por perplejidad

CCNet [28] puntúa cada página con un modelo de n-gramas entrenado sobre Wikipedia y conserva las de perplejidad baja; LLaMA usó este proceso para su parte de Common Crawl [25, §2.1]. La idea es que el texto parecido al de referencia recibe probabilidad alta. El caso de un modelo de unigramas permite analizar exactamente qué mide el filtro.

Definición 1.4 (Perplejidad de un documento). Sea q una distribución sobre $\mathcal{V}$ con $q(v) > 0$ para todo v (un modelo de unigramas de referencia) y sea $x = (x_1, \dots, x_T)$ un documento. La perplejidad de x según q es

$$\mathrm{PPL}_q(x) = \exp\Big(-\frac{1}{T}\sum_{t=1}^{T} \ln q(x_t)\Big).$$

La *distribución empírica* de x es $\hat{p}_x(v) = |\{t : x_t = v\}|/T$.

Proposición 1.5. *Para todo documento x,*

$$\ln \mathrm{PPL}_q(x) = H(\hat{p}_x, q) = H(\hat{p}_x) + D(\hat{p}_x \,\|\, q).$$

La proposición 1.5 muestra que el filtro premia dos cosas distintas: que el documento se parezca a la referencia (divergencia pequeña) y que el documento sea poco variado (entropía empírica pequeña). La segunda es indeseable.

Corolario 1.6 (El filtro premia la repetición). *Sea $q_{\max} = \max_v q(v)$. Para todo documento x, $\mathrm{PPL}_q(x) \geq 1/q_{\max}$, con igualdad si y solo si todos los tokens de x maximizan q.*

1.4 Clasificadores de calidad

Una alternativa es aprender qué es «calidad» a partir de ejemplos. Se toman como positivos documentos de una fuente de referencia (por ejemplo, páginas citadas desde Wikipedia) y como negativos páginas aleatorias de Common Crawl, y se entrena un clasificador lineal [10, §2.7].

Definición 1.8 (Filtro por clasificador). Sea $g\colon \mathcal{X} \to [0,1]$ un clasificador que estima la probabilidad de que un documento proceda de la fuente de referencia. El filtro con umbral τ conserva los documentos con $g(x) \geq \tau$. Subir τ aumenta la precisión y reduce la cobertura.

LLaMA entrenó un clasificador lineal que distingue las páginas usadas como referencia en Wikipedia de páginas aleatorias, y descartó las no clasificadas como referencia [25, §2.1]. FineWeb-Edu sustituye la fuente de referencia por anotaciones de un modelo grande: Llama-3-70B-Instruct puntuó el «valor educativo» de una muestra de páginas de 0 a 5, se ajustó una regresión sobre embeddings y se conservaron las páginas con puntuación al menos 3, unos 1,3 billones de tokens [20, §4].

Observación 1.9. «Calidad» es una elección, no una propiedad objetiva del texto. Un clasificador entrenado con Wikipedia como modelo de lo deseable tiende a descartar registros informales, variedades dialectales e idiomas con pocos recursos, y el modelo resultante hereda ese sesgo. Las reglas heurísticas tienen el mismo problema en menor grado: un umbral de longitud media de palabra pensado para el inglés no es neutral para otros idiomas.

1.5 Mezcla de fuentes y épocas

El corpus final combina varias fuentes (web, código, libros, artículos científicos, Wikipedia) con pesos fijados por experimentos.

Definición 1.10 (Mezcla y épocas). Sean m fuentes con U_i tokens únicos cada una y sean $w_i \geq 0$, $\sum_i w_i = 1$, los *pesos de muestreo*. Un entrenamiento de D tokens extrae $w_i D$ tokens de la fuente i; su número de *épocas* es

$$e_i = \frac{w_i D}{U_i}.$$

Si $e_i > 1$, la fuente se *sobremuestrea*: parte de su texto se ve varias veces.

Proposición 1.11. *Sea $E_{\max} > 0$. Existe una mezcla con $e_i \leq E_{\max}$ para todo i si y solo si $D \leq E_{\max} \sum_i U_i$.*

1.6 La escala de los datos

La cuadro 1.1 recoge la relación entre tokens de entrenamiento y parámetros de varios modelos con informe público. El cambio entre 2020 y 2023 se explica en los temas 4 y 5.

Cuadro 1.1: Parámetros, tokens de entrenamiento y tokens por parámetro.

Modelo (referencia)	N	D	D/N
GPT-3 [5]	$1{,}75 \cdot 10^{11}$	$3 \cdot 10^{11}$	1,7
Gopher [22]	$2{,}8 \cdot 10^{11}$	$3 \cdot 10^{11}$	1,1
Chinchilla [11]	$7 \cdot 10^{10}$	$1{,}4 \cdot 10^{12}$	20
LLaMA 7B [25]	$6{,}7 \cdot 10^{9}$	10^{12}	149
Llama 2 7B [26]	$6{,}74 \cdot 10^{9}$	$2 \cdot 10^{12}$	297
Llama 3 405B [7]	$4{,}05 \cdot 10^{11}$	$1{,}56 \cdot 10^{13}$	38,5

Tema 2

Deduplicación: MinHash y LSH

La web contiene muchas copias del mismo texto: páginas idénticas, plantillas, licencias que solo cambian en el año y el autor, artículos reproducidos con otro pie de página. Este tema estudia cómo detectarlas a escala de miles de millones de documentos. Los duplicados exactos se resuelven con funciones hash; los casi duplicados, con la similitud de Jaccard, las firmas MinHash y el *hashing* sensible a la localidad (LSH). Se sigue el capítulo 3 de Leskovec, Rajaraman y Ullman [15] (§3.1, §3.3 y §3.4), las clases «Locality-Sensitive Hashing I y II» del curso CS246 [14] y la sección 3 de la práctica de datos de CS336 [10]. MinHash procede de Broder [4].

2.1 Por qué deduplicar

Lee *et al.* [13] mostraron que eliminar duplicados de corpus estándar reduce la frecuencia con que el modelo reproduce texto literal del entrenamiento y permite alcanzar la misma calidad con menos pasos. Hay tres razones complementarias.

(i) *Cómputo.* Con un presupuesto $C \approx 6ND$ fijo (módulo 14), cada token repetido ocupa el lugar de uno nuevo.

(ii) *Memorización.* Lo que aparece cientos de veces se aprende literalmente y puede regurgitarse, incluidos datos personales o protegidos.

(iii) *Contaminación.* Si un documento de evaluación tiene copias en el corpus de entrenamiento, las métricas se inflan.

2.2 Duplicados exactos

Definición 2.1 (Huella). Sea $H\colon \Sigma^* \to \{0,1\}^\ell$ una función hash de ℓ bits (por ejemplo, SHA-1 o MD5). La *huella* de un documento normalizado x es $H(x)$. La deduplicación exacta conserva un documento por huella, con un recorrido y un diccionario de huellas: coste lineal en el tamaño del corpus.

Proposición 2.2 (Colisiones). *Supóngase que H asigna a cada uno de M documentos distintos un valor uniforme e independiente en $\{0,1\}^\ell$. El número esperado de pares de documentos distintos con la misma huella es $\binom{M}{2}2^{-\ell}$.*

Con $M = 10^9$ documentos hay $\binom{M}{2} \approx 5 \cdot 10^{17}$ pares: una huella de 64 bits produce en promedio 0,027 colisiones, y una de 128 bits, $1{,}5 \cdot 10^{-21}$. La deduplicación exacta también puede hacerse por líneas: se cuentan las apariciones de cada línea en el corpus y se eliminan las repetidas, lo que borra cabeceras, menús y pies de página [10, §3.1]. Llama 3 elimina las líneas que aparecen más de seis veces en cada grupo de 30 millones de documentos [7, §3.1.1].

2.3 Shingles y similitud de Jaccard

Dos páginas que solo difieren en la fecha tienen huellas distintas. Para detectar casi duplicados se representa cada documento por un conjunto y se mide el solapamiento [15, §3.1].

Definición 2.3 (Shingles). Sea $k \geq 1$. El conjunto de *k-shingles* de un documento de palabras $(u_1, \ldots, u_T)$ es $S_k = \{(u_t, \ldots, u_{t+k-1}) : 1 \leq t \leq T - k + 1\}$, el conjunto de grupos de k palabras consecutivas.

Definición 2.4 (Similitud de Jaccard). Sean A y B conjuntos finitos, no ambos vacíos. Su *similitud de Jaccard* es

$$J(A, B) = \frac{|A \cap B|}{|A \cup B|} \in [0, 1].$$

Vale 1 si y solo si $A = B$ y vale 0 si y solo si son disjuntos. La *distancia de Jaccard* $1 - J(A, B)$ es una distancia en el sentido métrico [15, cap. 3].

Comparar todos los pares de $M = 10^9$ documentos exige unos $5 \cdot 10^{17}$ cálculos de J, inabordable. Las dos secciones siguientes resuelven, respectivamente, el coste de almacenar los conjuntos y el de compararlos todos.

2.4 Firmas MinHash

Definición 2.6 (MinHash). Sea Ω un universo finito que contiene todos los shingles y sea π una permutación de Ω, identificada con una biyección $\pi \colon \Omega \to \{1, \ldots, |\Omega|\}$. El *minhash* de un conjunto no vacío $S \subseteq \Omega$ es $h_\pi(S) = \min_{x \in S} \pi(x)$. Con n permutaciones $\pi_1, \ldots, \pi_n$, la *firma* de S es el vector $\operatorname{sig}(S) = \big(h_{\pi_1}(S), \ldots, h_{\pi_n}(S)\big)$.

Teorema 2.7 (Propiedad fundamental de MinHash). *Si π es una permutación uniforme de Ω y $A, B \subseteq \Omega$ son no vacíos,*

$$\mathbb{P}\big(h_\pi(A) = h_\pi(B)\big) = J(A, B).$$

Corolario 2.8 (Estimador de Jaccard). *Sean $\pi_1, \ldots, \pi_n$ permutaciones uniformes e independientes y sea $\hat{J}$ la fracción de coordenadas en que coinciden* $\operatorname{sig}(A)$ *y* $\operatorname{sig}(B)$*. Entonces $n\hat{J} \sim \operatorname{Bin}(n, J)$, con $J = J(A, B)$; en particular,*

$$\mathbb{E}[\hat{J}] = J \qquad y \qquad \operatorname{Var}(\hat{J}) = \frac{J(1 - J)}{n}.$$

Con $n = 100$ funciones, la desviación típica es como mucho $\sqrt{0{,}25/100} = 0{,}05$. La firma sustituye un conjunto de miles de shingles por n enteros. En la práctica, las permutaciones se aproximan con funciones hash de una misma familia con semillas distintas [10, §3.2]: calcular $\min_{x \in S} h(x)$ requiere un único recorrido de S.

2.5 LSH por bandas

Definición 2.9 (LSH por bandas). Sea $n = br$. Se divide la firma en b bandas de r coordenadas consecutivas. Dos documentos son *candidatos* si sus firmas coinciden en todas las coordenadas de al menos una banda. En la práctica se asigna cada banda de cada documento a un cubo mediante una función hash, y solo se comparan los documentos que comparten algún cubo.

Proposición 2.10 (Curva en S). *Sean dos documentos con similitud de Jaccard s y firmas construidas con permutaciones independientes. La probabilidad de que sean candidatos es*

$$P(s) = 1 - \big(1 - s^r\big)^b.$$

La función P es creciente en $[0,1]$, con $P(0) = 0$ y $P(1) = 1$; vale $1/2$ en $s_{1/2} = (1 - 2^{-1/b})^{1/r}$ y, si $r \geq 2$ y $b \geq 2$, tiene un único punto de inflexión en $(0,1)$,

$$s^* = \Big(\frac{r-1}{rb-1}\Big)^{1/r}.$$

Cuando r es grande, $(r-1)/(rb-1) \to 1/b$, y se recupera la aproximación habitual del umbral, $s^* \approx (1/b)^{1/r}$ [15, §3.4]. Los pares con s muy por debajo del umbral casi nunca se comparan y los pares con s muy por encima casi siempre (figura 2.1).

Tema 3

Leyes de potencia y leyes de escalado de Kaplan

Kaplan *et al.* [12] observaron en 2020 que la pérdida de validación de un Transformer disminuye como una ley de potencia en cada uno de tres recursos, el número de parámetros N, el número de tokens D y el cómputo C, a lo largo de varios órdenes de magnitud. Este tema estudia primero las leyes de potencia como objeto matemático (por qué son las funciones invariantes por cambio de escala y cómo se ajustan) y después enuncia las leyes de Kaplan y su recomendación de reparto. Se sigue [12, §1.2, §4 y §6], la clase «Scaling laws» del curso CS324 [16] y la revisión de leyes de escalado de la práctica 3 de CS336 [9, §2].

3.1 Leyes de potencia

Definición 3.1 (Ley de potencia). Una función $f\colon (0,\infty)\to(0,\infty)$ es una *ley de potencia* si existen $c>0$ y $\alpha\in\mathbb{R}$ tales que $f(x)=c\,x^{-\alpha}$ para todo $x>0$. El número α es el *exponente*; si $\alpha>0$, f es decreciente.

La propiedad que distingue a las leyes de potencia es que no tienen escala característica: multiplicar x por un factor λ multiplica f por un factor que no depende del punto de partida.

Proposición 3.2 (Invariancia de escala). *Sea $f\colon(0,\infty)\to(0,\infty)$ continua. Existe una función g con $f(\lambda x)=g(\lambda)f(x)$ para todos $\lambda,x>0$ si y solo si f es una ley de potencia. En ese caso, $g(\lambda)=\lambda^{-\alpha}$.*

Corolario 3.3. *Si $L=cN^{-\alpha}$, cada vez que N se multiplica por λ la pérdida se multiplica por $\lambda^{-\alpha}$, sea cual sea N. En particular, la mejora relativa por cada duplicación de N es constante, $1-2^{-\alpha}$.*

3.2 Ajuste en escala log-log

Proposición 3.4. *$L=cN^{-\alpha}$ si y solo si $\log L=\log c-\alpha\log N$. En el plano $(u,v)=(\log N,\log L)$, una ley de potencia es una recta de pendiente $-\alpha$, en cualquier base del logaritmo.*

En consecuencia, el exponente se estima por regresión lineal de v sobre u (módulo 8).

Definición 3.5 (Ajuste de una ley de potencia). Dados entrenamientos (N_j,L_j), $j=1,\dots,m$, con $u_j=\log N_j$ y $v_j=\log L_j$, el ajuste por mínimos cuadrados en escala log-log es

la recta $v = b_0 + b_1 u$ con

$$b_1 = \frac{\sum_j (u_j - \bar{u})(v_j - \bar{v})}{\sum_j (u_j - \bar{u})^2}, \qquad b_0 = \bar{v} - b_1 \bar{u},$$

y la ley ajustada es $\hat{L}(N) = 10^{b_0} N^{b_1}$ (con $\log = \log_{10}$), de exponente $\hat{\alpha} = -b_1$.

Observación 3.6. El ajuste en escala logarítmica minimiza errores *relativos*, no absolutos, y da el mismo peso a cada orden de magnitud. Es lo adecuado cuando los entrenamientos están espaciados geométricamente, como es habitual. Extrapolar es otra cuestión: una recta ajustada en 10^7–10^9 parámetros y usada en 10^{11} supone que nada cambia en dos órdenes de magnitud sin observar.

3.3 Las leyes de Kaplan

Kaplan *et al.* entrenaron Transformers decoder-only sobre WebText2 y midieron la entropía cruzada de validación en nats por token sobre contextos de 1024 tokens [12, §2]. El número de parámetros N excluye los embeddings de vocabulario y de posición [12, §1.3].

Definición 3.7 (Leyes de Kaplan). Cuando el crecimiento de la pérdida está limitado por un único recurso, la pérdida sigue [12, ecuaciones (1.1)–(1.3)]

$$L(N) = \Big(\frac{N_c}{N}\Big)^{\alpha_N}, \qquad L(D) = \Big(\frac{D_c}{D}\Big)^{\alpha_D}, \qquad L(C_{\min}) = \Big(\frac{C_c}{C_{\min}}\Big)^{\alpha_C},$$

con los valores ajustados de la cuadro 3.1. La ley en N supone datos suficientes y entrenamiento hasta la convergencia; la ley en D, un modelo grande con parada temprana; la ley en $C_{\min}$, el mejor reparto posible del cómputo.

Cuadro 3.1: Constantes de las leyes de Kaplan *et al.* [12, §1.2]. Un PF-día son $10^{15} \cdot 86\,400 = 8{,}64 \cdot 10^{19}$ FLOP.

Ley	Exponente	Constante
$L(N)$	$\alpha_N \approx 0{,}076$	$N_c \approx 8{,}8 \cdot 10^{13}$ parámetros
$L(D)$	$\alpha_D \approx 0{,}095$	$D_c \approx 5{,}4 \cdot 10^{13}$ tokens
$L(C_{\min})$	$\alpha_C \approx 0{,}050$	$C_c \approx 3{,}1 \cdot 10^{8}$ PF-días

Las constantes dependen del tokenizador y de la distribución de los datos: no se trasladan a otro corpus. Los exponentes son más estables, y son lo que interesa para decidir cómo crecer.

3.4 Ley conjunta y sobreajuste

Cuando ambos recursos son finitos, Kaplan *et al.* proponen [12, ecuación (1.5) y §4]

$$L(N, D) = \Big[\Big(\frac{N_c}{N}\Big)^{\alpha_N/\alpha_D} + \frac{D_c}{D}\Big]^{\alpha_D}. \tag{3.1}$$

Proposición 3.9. *La función* (3.1) *es decreciente en N y en D, y sus límites son las leyes de un solo recurso:*

$$\lim_{D\to\infty} L(N, D) = L(N), \qquad \lim_{N\to\infty} L(N, D) = L(D).$$

La forma (3.1) permite cuantificar el sobreajuste: la penalización relativa por datos finitos depende de N y D solo a través de $N^{\alpha_N/\alpha_D}/D$ (ejercicio 3.5). De ahí la regla empírica de Kaplan *et al.* para que el sobreajuste quede dentro de la variación entre semillas, $D \gtrsim 5 \cdot 10^3\, N^{0{,}74}$ [12, ecuación (4.4)]. Con $N = 10^9$, la regla pide $D \gtrsim 2{,}3 \cdot 10^{10}$ tokens.

3.5 El reparto de Kaplan

El cómputo de entrenamiento es $C \approx 6ND$ FLOP (módulo 14, tema 6; [12, §2.1]). Kaplan *et al.* estudian cómo repartir un presupuesto creciente y concluyen que el tamaño óptimo crece como [12, ecuación (6.1)]

$$N_{\text{opt}} \propto C^{0{,}73},$$

y los datos, como $D \propto C^{0{,}27}$ [12, §1.1]. Con diez veces más cómputo, el modelo debería crecer $10^{0{,}73} = 5{,}4$ veces y los datos solo $10^{0{,}27} = 1{,}9$ veces. GPT-3 siguió esa pauta: $N = 1{,}75 \cdot 10^{11}$ parámetros y $D = 3 \cdot 10^{11}$ tokens [5], es decir, $C \approx 6 \cdot 1{,}75 \cdot 10^{11} \cdot 3 \cdot 10^{11} = 3{,}15 \cdot 10^{23}$ FLOP, unos 3650 PF-días, con menos de dos tokens por parámetro. El tema 4 muestra que esta recomendación no se sostiene.

Observación 3.10 (Hace falta un suelo). Una ley de potencia pura tiende a 0 cuando el recurso crece. Pero la pérdida de un modelo es una entropía cruzada $H(p, q)$, y por la desigualdad de Gibbs (módulo 1, tema 5) nunca baja de la entropía $H(p)$ del propio texto. Toda ley realista debe incluir, por tanto, un término constante irreducible; es lo que hace la ley de Chinchilla del tema 4.

Tema 4

El reparto óptimo del cómputo: Chinchilla

Hoffmann *et al.* [11] entrenaron más de 400 modelos, de menos de 70 millones a más de 16 000 millones de parámetros, con entre unos 5 000 millones y más de 400 000 millones de tokens, y concluyeron que, a cómputo fijo, el número de parámetros y el de tokens deben crecer aproximadamente al mismo ritmo. Este tema plantea el reparto como un problema de optimización con restricciones, lo resuelve en forma cerrada con multiplicadores de Lagrange para la ley paramétrica de Hoffmann *et al.* y extrae sus consecuencias. Se sigue [11, §3.3, ecuaciones (2) y (4), y apéndice D.2], la revisión de la práctica 3 de CS336 [9, §2] y, para la forma convexa del problema, la programación geométrica de Boyd y Vandenberghe [3, §4.5]. Los métodos empíricos con que se estiman las constantes se estudian en el tema 5.

4.1 El problema del reparto

Definición 4.1 (Reparto óptimo del cómputo). Sea $L(N, D)$ la pérdida esperada de un modelo de N parámetros entrenado con D tokens y sea $C > 0$ un presupuesto de FLOP. El *reparto óptimo* es la solución de

$$\min_{N,D>0} \; L(N, D) \qquad \text{sujeto a} \qquad 6ND = C.$$

Sus componentes se denotan $N_{\text{opt}}(C)$ y $D_{\text{opt}}(C)$. La *frontera eficiente* es la curva $C \mapsto (N_{\text{opt}}(C), D_{\text{opt}}(C))$.

4.2 La ley paramétrica de Chinchilla

Definición 4.2 (Ley de Chinchilla). La ley paramétrica de Hoffmann *et al.* [11, ecuación (2)] es

$$L(N, D) = E + \frac{A}{N^{\alpha}} + \frac{B}{D^{\beta}}, \qquad A, B, \alpha, \beta, E > 0.$$

El ajuste del enfoque 3 da [11, apéndice D.2]

$$E = 1{,}69, \qquad A = 406{,}4, \qquad B = 410{,}7, \qquad \alpha = 0{,}34, \qquad \beta = 0{,}28.$$

Los tres términos tienen una interpretación directa [11, §3.3]. E es la pérdida de un predictor ideal: la entropía del texto, que ningún modelo puede rebajar (desigualdad de Gibbs). A/N^{α} es el exceso de un modelo de tamaño finito entrenado con datos ilimitados. B/D^{β} es el exceso por entrenar con un número finito de tokens, es decir, de pasos de optimización. A diferencia de las leyes de Kaplan, la ley tiene un suelo: con $N = D = 10^{30}$, los dos términos de error valen $2{,}6 \cdot 10^{-8}$ y $1{,}6 \cdot 10^{-6}$, y $L \approx E$.

4.3 La solución en forma cerrada

Teorema 4.3 (Frontera eficiente de Chinchilla). *Sea L como en la definición 4.2. Para cada $C > 0$, el problema de la definición 4.1 tiene una única solución,*

$$N_{\text{opt}}(C) = G\Big(\frac{C}{6}\Big)^{a}, \qquad D_{\text{opt}}(C) = G^{-1}\Big(\frac{C}{6}\Big)^{b},$$

con

$$G = \Big(\frac{\alpha A}{\beta B}\Big)^{\frac{1}{\alpha+\beta}}, \qquad a = \frac{\beta}{\alpha+\beta}, \qquad b = \frac{\alpha}{\alpha+\beta}.$$

Corolario 4.4 (Equilibrio de los términos de error). *En el óptimo, $\dfrac{A/N_{\text{opt}}^{\alpha}}{B/D_{\text{opt}}^{\beta}} = \dfrac{\beta}{\alpha}$. Con los valores de Chinchilla, el término de datos es $\alpha/\beta = 1{,}214$ veces el de parámetros.*

Corolario 4.5 (Exponentes del reparto). *$N_{\text{opt}} \propto C^{a}$ y $D_{\text{opt}} \propto C^{b}$, con $a + b = 1$. Con los valores de Chinchilla, $a = 0{,}28/0{,}62 = 0{,}452$, $b = 0{,}548$ y $G = 1{,}345$.*

El artículo da $a \approx 0{,}46$ para este enfoque y $a \approx 0{,}50$ y $0{,}49$ para los otros dos [11, tabla 2], frente al 0,73 de Kaplan *et al.*: el modelo y los datos crecen, aproximadamente, como $\sqrt{C}$.

Corolario 4.6 (Pérdida en la frontera). *Sea $\gamma = \alpha\beta/(\alpha+\beta)$. Existe $K > 0$ tal que $L(N_{\text{opt}}(C), D_{\text{opt}}(C)) - E = K\,C^{-\gamma}$. Con los valores de Chinchilla, $\gamma = 0{,}1535$.*

El corolario 4.6 reconcilia las dos líneas de trabajo: el exceso sobre la entropía irreducible decae como una ley de potencia en el cómputo, como observaron Kaplan *et al.*, aunque el reparto que la produce es otro.

4.4 Gopher frente a Chinchilla

4.5 La regla de los 20 tokens

Los enfoques 1 y 2 de Hoffmann *et al.* conducen en la práctica a unos 20 tokens por parámetro: la tabla 3 de [11] asigna $20{,}2 \cdot 10^{9}$ tokens a un modelo de 10^{9} parámetros y $205{,}1 \cdot 10^{9}$ a uno de 10^{10}.

Proposición 4.8 (Regla de proporción fija). *Si se impone $D = \rho N$ con $\rho > 0$ fijo, la restricción $6ND = C$ da $N = \sqrt{C/(6\rho)}$ y $D = \rho N$. Con $\rho = 20$, $N = \sqrt{C/120}$.*

Por ejemplo, $C = 1{,}2 \cdot 10^{20}$ da $N = 10^{9}$ y $D = 2 \cdot 10^{10}$, de acuerdo con la tabla 3 de [11]. El ajuste del enfoque 3 da, en cambio, unos 63 tokens por parámetro a 10^{22} FLOP (ejercicio 4.3); la discrepancia entre los enfoques se discute en el tema 5.

4.6 El mínimo es plano

Proposición 4.9 (Coste de desviarse del óptimo). *Sobre la restricción $6ND = C$, sea $\delta = \ln(N/N_{\text{opt}})$ la desviación logarítmica del tamaño, y sean $T_N = AN_{\text{opt}}^{-\alpha}$ y $T_D = BD_{\text{opt}}^{-\beta}$ los términos de error en el óptimo. El exceso de pérdida es*

$$\Delta(\delta) = T_N\big(e^{-\alpha\delta} - 1\big) + T_D\big(e^{\beta\delta} - 1\big) = \tfrac{1}{2}\big(\alpha^2 T_N + \beta^2 T_D\big)\delta^2 + O(\delta^3).$$

4.7 Forma convexa del problema

El problema de la definición 4.1 con la ley de Chinchilla es un *programa geométrico* [3, §4.5]: el objetivo $AN^{-\alpha} + BD^{-\beta}$ es un posinomio (suma de monomios con coeficientes positivos y exponentes reales) y la restricción $6ND \leq C$ es una cota sobre un monomio. Con el cambio de variables $u = \ln N$, $v = \ln D$, el objetivo pasa a ser $Ae^{-\alpha u} + Be^{-\beta v}$, convexo, y la restricción, $u + v \leq \ln(C/6)$, afín. El problema transformado es convexo, y sus condiciones de Karush–Kuhn–Tucker son suficientes para el óptimo global (ejercicio 4.7).

Tema 5

Estimación empírica, inferencia y datos limitados

El tema 4 supuso conocida la ley $L(N, D)$. Este tema estudia cómo se estima a partir de entrenamientos pequeños, por qué Kaplan *et al.* y Hoffmann *et al.* llegaron a conclusiones distintas y cómo cambia el reparto cuando el objetivo no es solo el cómputo de entrenamiento: cuando el modelo va a servirse durante mucho tiempo y cuando los datos únicos escasean. Se sigue [11, §3.1–3.3 y tabla 2], la práctica 3 de CS336 [9, §2.1], Muennighoff *et al.* [18, §3.1 y apéndice A], Sardana *et al.* [23] y la réplica de Besiroglu *et al.* [2].

5.1 Tres métodos de estimación

Hoffmann *et al.* estimaron la frontera eficiente por tres caminos independientes.

Definición 5.1 (Enfoque 1: envolvente de las curvas de entrenamiento). Se fijan varios tamaños de modelo y, para cada uno, se entrenan varias longitudes de entrenamiento con un calendario de tasa de aprendizaje ajustado a cada longitud [11, §3.1]. Cada curva de pérdida, reexpresada en función de los FLOP consumidos, $6N \cdot$ (tokens vistos), aporta puntos (C, L). Para cada C se toma el entrenamiento de menor pérdida, y los pares $(C, N_{\text{opt}}(C))$ resultantes se ajustan con una ley de potencia.

Definición 5.2 (Enfoque 2: perfiles IsoFLOP). Se fijan varios presupuestos C_i y, para cada uno, se entrenan modelos de distintos tamaños N_{ij} con $D_{ij} = C_i/(6N_{ij})$ tokens. La curva $N \mapsto L$ a C_i fijo es el *perfil IsoFLOP* de C_i. Hoffmann *et al.* ajustan una parábola en $\log N$ a cada perfil y toman su vértice como $N_{\text{opt}}(C_i)$ [11, §3.2]; la práctica de CS336 recomienda tomar sin más el entrenamiento de menor pérdida [9, §2.1]. Después se ajustan $N_{\text{opt}} \propto C^a$ y $D_{\text{opt}} \propto C^b$ por mínimos cuadrados en escala log-log.

La forma en U de cada perfil tiene una explicación sencilla [9, §2.1]: un modelo muy pequeño no puede ajustar los datos por mucho que se entrene, y uno muy grande agota el presupuesto en pocos pasos de optimización. La figura 5.1 muestra el esquema.

Proposición 5.3 (Vértice de un perfil). *Sean tres entrenamientos de un perfil IsoFLOP con $x_j = \log_{10} N_j$ equiespaciados, $x_1 = x_2 - h$, $x_3 = x_2 + h$, y pérdidas L_1, L_2, L_3 con $\Delta_2 = L_1 - 2L_2 + L_3 > 0$. La parábola que pasa por los tres puntos tiene su vértice en*

$$x^* = x_2 - \frac{h\,(L_3 - L_1)}{2\Delta_2}, \qquad L^* = L_2 - \frac{(L_3 - L_1)^2}{8\Delta_2}.$$

Definición 5.4 (Enfoque 3: ajuste paramétrico). Se ajustan directamente las cinco cons-

tantes de la definición 4.2 a todos los entrenamientos (N_j, D_j, L_j) minimizando

$$\sum_j \mathrm{Huber}_\delta\Big(\ln \hat{L}(N_j, D_j) - \ln L_j\Big), \qquad \mathrm{Huber}_\delta(z) = \begin{cases} z^2/2, & |z| \le \delta, \\ \delta(|z| - \delta/2), & |z| > \delta, \end{cases}$$

con $\delta = 10^{-3}$, mediante L-BFGS desde una rejilla de puntos iniciales [11, §3.3]. La pérdida de Huber crece linealmente para residuos grandes, lo que reduce el peso de los entrenamientos anómalos.

Cuadro 5.1: Exponentes de la frontera eficiente, $N_{\mathrm{opt}} \propto C^a$ y $D_{\mathrm{opt}} \propto C^b$, según [11, tabla 2].

Método	a	b
Enfoque 1: envolvente de las curvas	0,50	0,50
Enfoque 2: perfiles IsoFLOP	0,49	0,51
Enfoque 3: ajuste paramétrico	0,46	0,54
Kaplan *et al.* [12]	0,73	0,27

5.2 Por qué difieren Kaplan y Chinchilla

Hoffmann *et al.* atribuyen la mayor parte de la discrepancia a un detalle experimental: Kaplan *et al.* usaron el mismo número de tokens y el mismo calendario de tasa de aprendizaje para todos los modelos [11, §2 y apéndice B]. Con un decaimiento coseno (módulo 7) pensado para un entrenamiento largo, las pérdidas intermedias de un entrenamiento corto se miden con la tasa de aprendizaje todavía alta, y son peores de lo que serían con un calendario ajustado a esa longitud. Eso penaliza a los entrenamientos con pocos tokens de forma desigual y sesga la conclusión hacia modelos grandes con pocos datos.

Observación 5.5 (Las constantes son frágiles). Besiroglu *et al.* [2] reconstruyeron los datos del enfoque 3 a partir de las figuras del artículo y concluyeron que las constantes publicadas no ajustan bien esos datos, que los intervalos de confianza declarados son demasiado estrechos y que un nuevo ajuste da resultados compatibles con los enfoques 1 y 2. La conclusión cualitativa, $a \approx b \approx 1/2$, es robusta; las constantes exactas, no. Por eso los ejemplos numéricos de estos apuntes deben leerse como ilustración del método.

5.3 El coste de la inferencia

El reparto de Chinchilla minimiza el cómputo de *entrenamiento*. Pero un modelo se entrena una vez y se usa muchas veces, y cada token generado cuesta unos $2N$ FLOP (módulo 14, tema 6).

Definición 5.6 (Coste total). Si un modelo de N parámetros se entrena con D tokens y se usa para procesar T_{inf} tokens de inferencia, su coste total es

$$C_{\mathrm{tot}} = 6ND + 2NT_{\mathrm{inf}}.$$

Proposición 5.7 (Amortización del sobreentrenamiento). *Sean dos configuraciones con la misma pérdida,* (N_1, D_1) *y* (N_2, D_2)*, con* $N_2 < N_1$ *y* $6N_2D_2 > 6N_1D_1$*. La configuración*

pequeña tiene menor coste total si y solo si

$$T_{\text{inf}} > T^* = \frac{6(N_2 D_2 - N_1 D_1)}{2(N_1 - N_2)}.$$

Con la ley de Chinchilla, el sobrecoste relativo de entrenamiento de reducir el modelo en un factor fijo no depende del presupuesto.

Proposición 5.8 (Sobrecoste de reducir el modelo). *Sea* $(N_{\text{opt}}, D_{\text{opt}})$ *el óptimo del teorema 4.3 para el presupuesto* C *y sea* $0 < \kappa < 1$ *con* $(\kappa^{-\alpha} - 1)\beta/\alpha < 1$. *El modelo de* κN_{opt} *parámetros alcanza la pérdida óptima con* $D_2 = D_{\text{opt}}\big(1 - (\kappa^{-\alpha} - 1)\beta/\alpha\big)^{-1/\beta}$ *tokens, y su cómputo de entrenamiento es*

$$\frac{C_2}{C} = \kappa\Big(1 - \frac{\beta}{\alpha}\big(\kappa^{-\alpha} - 1\big)\Big)^{-1/\beta},$$

independiente de C. *Si* $\kappa \le (1 + \alpha/\beta)^{-1/\alpha}$, *ningún número de tokens iguala la pérdida óptima.*

Esta es la lógica de los modelos abiertos recientes. LLaMA entrenó un modelo de $7 \cdot 10^9$ parámetros con 10^{12} tokens, más de siete veces lo que daría la regla de los 20 tokens, y observó que su rendimiento seguía mejorando; sus autores señalan que la regla de Hoffmann *et al.* ignora el presupuesto de inferencia [25, §1]. Llama 2 7B duplicó esa cifra (cuadro 1.1). Sardana *et al.* [23] incorporan la demanda de inferencia a la ley de escalado y concluyen que, con una demanda del orden de mil millones de peticiones, conviene entrenar modelos más pequeños y durante más tiempo que el óptimo de Chinchilla.

5.4 Datos limitados: el valor de repetir

Si cada modelo pide billones de tokens, el texto único de calidad se agota. Muennighoff *et al.* [18] estudiaron qué vale un token visto por segunda, tercera o k-ésima vez.

Definición 5.10 (Datos efectivos). Sea un entrenamiento que recorre U tokens únicos durante $1 + R$ épocas, es decir, con $R \ge 0$ repeticiones. Los *datos efectivos* son [18, §3.1, ecuación (5)]

$$D' = U + U\,R^*\big(1 - e^{-R/R^*}\big),$$

donde $R^* > 0$ es una constante ajustada; Muennighoff *et al.* obtienen $R^* \approx 15{,}4$ [18, apéndice A]. Se usa D' en lugar de D en la ley de Chinchilla.

Proposición 5.11. *Como función de* $R \ge 0$, D' *es creciente y cóncava,* $D'(0) = U$, $D' < U(1 + R)$ *para todo* $R > 0$ *y* $D' \to U(1 + R^*)$ *cuando* $R \to \infty$.

El primer token repetido vale casi lo mismo que uno nuevo, pero el valor marginal decae exponencialmente con escala R^*: por muchas épocas que se hagan, un corpus de U tokens nunca equivale a más de unos $16{,}4\,U$ tokens nuevos. Con cuatro épocas ($R = 3$), $D' = U(1 + 15{,}39 \cdot 0{,}1772) = 3{,}73\,U$, el 93 % de los $4U$ tokens procesados. Los experimentos del artículo muestran que repetir hasta unas cuatro épocas apenas cambia la pérdida respecto de usar datos nuevos [18].

Tema 6

Capacidades emergentes y contexto largo

Las leyes de escalado predicen bien la pérdida de preentrenamiento. Este tema trata dos cuestiones en las que la escala produce efectos menos regulares. La primera es la aparición aparentemente brusca de capacidades al crecer el modelo, y si esa brusquedad es una propiedad del modelo o de la forma de medir. La segunda es la ampliación de la ventana de contexto, que tropieza con el coste cuadrático de la atención, con la memoria de la caché de claves y valores y con la codificación de posiciones no vistas en el entrenamiento. Se sigue a Wei *et al.* [27], Schaeffer, Miranda y Koyejo [24] y la clase «Capabilities» de CS324 [16] para la primera parte, y a Chen *et al.* [6], el informe de Llama 3 [7, §3.4.2] y Liu *et al.* [17] para la segunda; el modelo probabilístico de las respuestas usa la independencia de Bertsekas y Tsitsiklis [1, §1.5].

6.1 Capacidades emergentes

Definición 6.1 (Capacidad emergente; Wei *et al.* [27]). Una capacidad es *emergente* si no está presente en los modelos pequeños de una familia y sí en los grandes, de modo que no puede predecirse extrapolando el rendimiento de los primeros.

Los ejemplos habituales son tareas de aritmética de varias cifras y algunas tareas del conjunto BIG-Bench, en las que la exactitud permanece cerca del azar en varios órdenes de magnitud de cómputo y después sube con rapidez [27]. Si fueran saltos genuinos, la planificación de un modelo grande no podría apoyarse en experimentos pequeños, que es justo lo que hacen los temas 3 a 5.

6.2 El argumento de la métrica

Schaeffer, Miranda y Koyejo [24] proponen que muchas de esas discontinuidades las crea la métrica. Supóngase que una respuesta tiene k tokens y que el modelo acierta cada uno con probabilidad p, de forma independiente [1, §1.5]. La *exactitud exacta* (todo o nada) es la probabilidad de acertar los k:

$$\mathbb{P}(\text{respuesta correcta}) = p^k.$$

Una mejora gradual de p se convierte en un cambio brusco de p^k cuando k es grande:

p	0,5	0,8	0,9	0,95	0,99
p^5	0,031	0,328	0,590	0,774	0,951
p^{10}	0,001	0,107	0,349	0,599	0,904

Pasar de $p = 0{,}8$ a $p = 0{,}95$ lleva la exactitud de 0,107 a 0,599 con $k = 10$. En cambio, el número esperado de tokens correctos, kp, crece linealmente con p.

La pérdida por token sí sigue una ley de potencia suave en N. La proposición siguiente muestra qué forma adopta entonces la exactitud exacta.

Proposición 6.2 (Exactitud exacta bajo una ley de potencia). *Supóngase que la pérdida por token de la respuesta es $\ell(N) = (N_1/N)^\theta$, con $N_1, \theta > 0$, que $p(N) = e^{-\ell(N)}$ y que la respuesta tiene k tokens. Sea $x = \ln N$. Entonces:*

(i) $A(N) = p(N)^k = \exp\!\big(-e^{-\theta(x-x_0)}\big)$, *con $x_0 = \ln N_1 + (\ln k)/\theta$: una curva de Gompertz en $\ln N$, creciente de 0 a 1;*

(ii) su único punto de inflexión está en $N_0 = N_1 k^{1/\theta}$, donde $A = 1/e$;

(iii) en cambio, $-\ln A(N) = k\,\ell(N)$ es una ley de potencia en N, una recta en escala log-log.

La proposición 6.2 contiene el argumento completo. La capacidad «aparece» en torno a $N_0 = N_1 k^{1/\theta}$, y el umbral se desplaza hacia modelos mayores cuanto más larga es la respuesta exigida, aunque la mejora por token sea la misma. Con pocos tamaños de modelo y escala lineal en el eje vertical, la curva parece un escalón (figura 6.1); con la métrica continua $-\ln A$, es una recta en escala log-log.

Observación 6.3 (El debate sigue abierto). El argumento explica muchas discontinuidades observadas, y Schaeffer *et al.* las hacen desaparecer cambiando la métrica o aumentando la resolución estadística [24]. No demuestra que no existan cambios cualitativos con la escala: muestra que una curva brusca en una métrica de todo o nada no es, por sí sola, prueba de ellos. Lo que sí está claro es que la pérdida de preentrenamiento es predecible y el rendimiento en una tarea concreta lo es mucho menos.

6.3 El coste del contexto largo

La ventana de contexto ha crecido de 2048 tokens en GPT-3 [5] a 128K (131 072) en Llama 3 [7]; el informe de Gemini 1.5 describe ventanas de millones de tokens, con recuperación casi perfecta hasta al menos 10 millones, y cita 200 000 tokens para Claude 3.0 [8]. Hay tres obstáculos.

Cómputo de la atención

Por el módulo 14 (tema 6), una pasada hacia delante cuesta por token unos $2N + 4Ln_{\text{ctx}}d$ FLOP, donde L es el número de bloques y d la dimensión del modelo. El segundo término procede de los productos $QK^\top$ y AV y crece linealmente con el contexto; sobre una secuencia completa, el coste de la atención crece como n_{ctx}^2. Pasar de 4096 a 10^6 tokens multiplica ese término total por $(10^6/4096)^2 \approx 59\,600$.

Memoria de la caché KV

Proposición 6.5 (Tamaño de la caché KV). *Un Transformer de L bloques con h_{kv} cabezas de claves y valores de dimensión d_k, que almacena cada número con β_b bytes, necesita para una secuencia de n tokens una caché de*

$$M_{KV} = 2\,L\,n\,h_{kv}\,d_k\,\beta_b \ \text{bytes}.$$

Para LLaMA-2 7B, con atención multicabeza estándar ($h_{kv} = 32$, $d_k = 128$) en bf16 ($\beta_b = 2$), son $2 \cdot 32 \cdot 4096 \cdot 2 = 524\,288$ bytes, 0,5 MiB por token: 2 GiB con 4096 tokens y 64 GiB con 131 072, más que la memoria de una GPU de 40 GiB para una sola secuencia. La atención con consultas agrupadas (GQA, módulo 18) reduce h_{kv}: con 8 cabezas de claves y valores, la caché se divide por 4. La caché domina el coste de la inferencia con contexto largo (módulo 19).

6.4 Posiciones no vistas: interpolación de RoPE

En RoPE (módulo 14, tema 4), el par de coordenadas i de la consulta y la clave en la posición m se rota un ángulo $m\theta_i$, con $\theta_i = \text{base}^{-2i/d_k}$ para $i = 0, \ldots, d_k/2 - 1$, y el producto escalar resultante depende solo de la diferencia de posiciones. Un modelo entrenado con contexto L_{tr} solo ha visto ángulos en $[0, L_{\text{tr}}\theta_i)$. Para las frecuencias bajas, ese intervalo es una fracción pequeña de una vuelta, y las posiciones $m \geq L_{\text{tr}}$ producen ángulos que el modelo nunca ha visto.

Definición 6.6 (Interpolación de posiciones; Chen *et al.* [6]). Sea $L' > L_{\text{tr}}$ el nuevo contexto. La *interpolación de posiciones* sustituye la posición m por $m\,L_{\text{tr}}/L'$ en todas las rotaciones de RoPE, y se ajusta brevemente el modelo con secuencias de longitud L'.

Proposición 6.7. *Con interpolación de posiciones, para toda posición $0 \leq m < L'$ y toda frecuencia i, el ángulo de rotación está en $[0, L_{\text{tr}}\theta_i)$, el rango visto en el entrenamiento. El producto escalar rotado entre las posiciones m y n sigue dependiendo solo de $m - n$, ahora a través de $(m - n)L_{\text{tr}}/L'$.*

El precio es la resolución: dos posiciones contiguas quedan a L_{tr}/L' de distancia en lugar de 1. Las variantes posteriores reparten el reescalado entre frecuencias (YaRN [21]) o aumentan la base de RoPE; Llama 3 usa base 500 000 [7, §3.2].

6.5 La receta habitual y sus límites

Como el coste de la atención crece con el contexto, casi todo el preentrenamiento se hace con secuencias cortas y se añade una fase final con documentos largos. Llama 3 amplió su ventana de 8K a 128K tokens en seis etapas, con unos 800 000 millones de tokens en total [7, §3.4.2]. Aceptar un contexto largo no equivale a usarlo bien: Liu *et al.* [17] observaron que los modelos aprovechan peor la información situada en el centro de un contexto largo que la situada al principio o al final.

Bibliografía

[1] D. P. Bertsekas y J. N. Tsitsiklis. *Introduction to Probability*, 2.ª ed. Athena Scientific, 2008. Texto del curso 6.041 del MIT. §1.5, «Independence».

[2] T. Besiroglu, E. Erdil, M. Barnett y J. You. Chinchilla scaling: A replication attempt. Prepublicación, 2024. https://arxiv.org/abs/2404.10102

[3] S. Boyd y L. Vandenberghe. *Convex Optimization*. Cambridge University Press, 2004. §4.5 «Geometric programming» y §5.5 «Optimality conditions». https://web.stanford.edu/~boyd/cvxbook/

[4] A. Z. Broder. On the resemblance and containment of documents. En *Proceedings of Compression and Complexity of Sequences*, IEEE, 1997.

[5] T. B. Brown *et al.* Language models are few-shot learners. En *Advances in Neural Information Processing Systems 33 (NeurIPS)*, 2020. https://arxiv.org/abs/2005.14165

[6] S. Chen, S. Wong, L. Chen e Y. Tian. Extending context window of large language models via positional interpolation. Prepublicación, 2023. https://arxiv.org/abs/2306.15595

[7] A. Dubey *et al.* (Llama Team, Meta). The Llama 3 herd of models. Prepublicación, 2024. https://arxiv.org/abs/2407.21783

[8] Gemini Team, Google. Gemini 1.5: Unlocking multimodal understanding across millions of tokens of context. Informe técnico, 2024. https://arxiv.org/abs/2403.05530

[9] T. Hashimoto y P. Liang. CS336: Language Modeling from Scratch. Stanford University, primavera de 2026. Práctica 3, «Scaling Laws», versión 26.0.5. https://github.com/stanford-cs336/assignment3-scaling

[10] T. Hashimoto y P. Liang. CS336: Language Modeling from Scratch. Stanford University, primavera de 2026. Práctica 4, «Data: Filtering Language Modeling Data», versión 26.0.1. https://github.com/stanford-cs336/assignment4-data

[11] J. Hoffmann, S. Borgeaud, A. Mensch *et al.* Training compute-optimal large language models. En *Advances in Neural Information Processing Systems 35 (NeurIPS)*, 2022. https://arxiv.org/abs/2203.15556

[12] J. Kaplan, S. McCandlish, T. Henighan, T. B. Brown, B. Chess, R. Child, S. Gray, A. Radford, J. Wu y D. Amodei. Scaling laws for neural language models. Prepublicación, 2020. https://arxiv.org/abs/2001.08361

[13] K. Lee, D. Ippolito, A. Nystrom, C. Zhang, D. Eck, C. Callison-Burch y N. Carlini. Deduplicating training data makes language models better. En *Proceedings of ACL*, 2022. https://arxiv.org/abs/2107.06499

[14] J. Leskovec. CS246: Mining Massive Data Sets. Stanford University, invierno de 2026. Clases «Locality-Sensitive Hashing I» y «Locality-Sensitive Hashing II». https://web.stanford.edu/class/cs246/

[15] J. Leskovec, A. Rajaraman y J. D. Ullman. *Mining of Massive Datasets*, 3.ª ed. Cambridge University Press, 2020. Cap. 3, «Finding Similar Items». http://infolab.stanford.edu/~ullman/mmds/ch3n.pdf

[16] P. Liang, T. Hashimoto y C. Ré. CS324: Large Language Models. Stanford University, invierno de 2022. Notas de las clases «Capabilities», «Data» y «Scaling laws». https://stanford-cs324.github.io/winter2022/lectures/

[17] N. F. Liu, K. Lin, J. Hewitt, A. Paranjape, M. Bevilacqua, F. Petroni y P. Liang. Lost in the middle: How language models use long contexts. *Transactions of the Association for Computational Linguistics*, 12, 2024. https://arxiv.org/abs/2307.03172

[18] N. Muennighoff, A. M. Rush, B. Barak, T. Le Scao, A. Piktus, N. Tazi, S. Pyysalo, T. Wolf y C. Raffel. Scaling data-constrained language models. En *Advances in Neural Information Processing Systems 36 (NeurIPS)*, 2023. https://arxiv.org/abs/2305.16264

[19] G. Penedo *et al.* The RefinedWeb dataset for Falcon LLM: Outperforming curated corpora with web data, and web data only. Prepublicación, 2023. https://arxiv.org/abs/2306.01116

[20] G. Penedo, H. Kydlíček, L. Ben Allal, A. Lozhkov, M. Mitchell, C. Raffel, L. Von Werra y T. Wolf. The FineWeb datasets: Decanting the web for the finest text data at scale. Prepublicación, 2024. https://arxiv.org/abs/2406.17557

[21] B. Peng, J. Quesnelle, H. Fan y E. Shippole. YaRN: Efficient context window extension of large language models. Prepublicación, 2023. https://arxiv.org/abs/2309.00071

[22] J. W. Rae *et al.* Scaling language models: Methods, analysis & insights from training Gopher. Prepublicación, 2021. https://arxiv.org/abs/2112.11446

[23] N. Sardana, J. Portes, S. Doubov y J. Frankle. Beyond Chinchilla-optimal: Accounting for inference in language model scaling laws. Prepublicación, 2023. https://arxiv.org/abs/2401.00448

[24] R. Schaeffer, B. Miranda y S. Koyejo. Are emergent abilities of large language models a mirage? En *Advances in Neural Information Processing Systems 36 (NeurIPS)*, 2023. https://arxiv.org/abs/2304.15004

[25] H. Touvron *et al.* LLaMA: Open and efficient foundation language models. Prepublicación, 2023. https://arxiv.org/abs/2302.13971

[26] H. Touvron *et al.* Llama 2: Open foundation and fine-tuned chat models. Prepublicación, 2023. https://arxiv.org/abs/2307.09288

[27] J. Wei, Y. Tay, R. Bommasani, C. Raffel, B. Zoph, S. Borgeaud, D. Yogatama, M. Bosma, D. Zhou, D. Metzler, E. H. Chi, T. Hashimoto, O. Vinyals, P. Liang, J. Dean y W. Fedus. Emergent abilities of large language models. *Transactions on Machine Learning Research*, 2022. https://arxiv.org/abs/2206.07682

[28] G. Wenzek, M.-A. Lachaux, A. Conneau, V. Chaudhary, F. Guzmán, A. Joulin y E. Grave. CCNet: Extracting high quality monolingual datasets from web crawl data. En *Proceedings of LREC*, 2020. https://arxiv.org/abs/1911.00359